

Data Structures - Practice Questions & Answers

Q1. What is a Data Structure? Give 3 real-life examples.

A Data Structure is a way of organizing, storing, and managing data in a computer so that it can be accessed and modified efficiently. It defines the relationship between data elements and the operations that can be performed on them (such as insertion, deletion, searching, and traversal).

Real-life examples:

1. A library catalog system - organizes books (data) so they can be searched and retrieved quickly, similar to how a database index works.
2. A to-do list / queue at a ticket counter - people are served in the order they arrive, similar to a Queue data structure (FIFO).
3. A stack of plates - plates are added and removed from the top only, similar to a Stack data structure (LIFO).

Q2. Differentiate between Linear and Non-linear Data Structures.

Linear Data Structure: Elements are arranged in a sequential order, one after another. Each element (except the first and last) has exactly one predecessor and one successor. Examples: Array, Linked List, Stack, Queue. Traversal is done in a single run/pass and memory utilization is generally straightforward.

Non-linear Data Structure: Elements are not arranged sequentially; instead they are connected in a hierarchical or networked manner, where one element can be connected to multiple elements. Examples: Tree, Graph. Traversal usually requires multiple passes or special algorithms (like DFS/BFS) and memory utilization is comparatively more efficient for representing complex relationships.

Q3. Explain the internal structure of an Array with a memory diagram.

An Array is a linear data structure that stores a fixed number of elements of the same data type in contiguous memory locations. Each element can be accessed directly using its index, and the address of any element can be calculated using the formula:

Address of $A[i] = \text{Base Address} + (i \times \text{Size of each element})$

Memory diagram for an integer array $A[5] = \{10, 20, 30, 40, 50\}$, assuming Base Address = 1000 and each int takes 4 bytes:

Index: $A[0]$ $A[1]$ $A[2]$ $A[3]$ $A[4]$

Value: 10 20 30 40 50

Address: 1000 1004 1008 1012 1016

Since the elements are stored in contiguous memory locations, array access ($A[i]$) takes constant time $O(1)$, as the address can be computed directly without traversal.

Q4. Write the algorithm and C program for Array insertion.

Algorithm (Insert an element at a given position in an array):

1. Start.
2. Input the array, its current size (n), the position (pos) and the value to insert.
3. If pos is not valid ($\text{pos} < 0$ or $\text{pos} > n$), display an error and stop.
4. Shift all elements from the last index up to pos one position to the right.
5. Insert the new value at index pos.
6. Increase the size of the array ($n = n + 1$).
7. Stop.

```
#include <stdio.h>

int main() {
    int arr[20] = {10, 20, 30, 40, 50};
    int n = 5, pos, value, i;

    printf("Enter position (0 to %d): ", n);
    scanf("%d", &pos);
    printf("Enter value to insert: ");
    scanf("%d", &value);

    if (pos < 0 || pos > n) {
        printf("Invalid position\n");
        return 0;
    }

    /* Shift elements to the right */
    for (i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }

    arr[pos] = value;
    n++;

    printf("Array after insertion: ");
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Q5. Explain Stack using LIFO concept with a diagram.

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle, meaning the last element inserted into the stack is the first one to be removed. Insertion and deletion happen only from one end, called the TOP of the stack.

The two main operations are:

Push - insert an element onto the top of the stack.

Pop - remove the element from the top of the stack.

Diagram showing a stack after pushing 10, 20, then 30 (30 was inserted last, so it is on top):

| 30 | <- TOP (last in, will be first out)

| 20 |

| 10 | <- BOTTOM (first in)

When a Pop operation is performed, 30 will be removed first since it was the last value pushed, which demonstrates the LIFO behavior.

Q6. What is the difference between Stack and Queue?

Stack: Follows LIFO (Last In, First Out) principle. Insertion and deletion both happen from the same end, called the TOP. Main operations are Push and Pop. Example use case: function call management (call stack), undo operations in editors.

Queue: Follows FIFO (First In, First Out) principle. Insertion happens at the REAR and deletion happens at the FRONT, i.e., two different ends. Main operations are Enqueue and Dequeue. Example use case: task scheduling, ticket counter lines, printer job management.

Q7. Write C code for Enqueue and Dequeue operations.

Enqueue inserts an element at the rear of the queue, and Dequeue removes an element from the front of the queue. Below is a simple array-based implementation:

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Queue is full\n");
        return;
    }
    if (front == -1) front = 0;
    rear++;
    queue[rear] = value;
    printf("%d enqueued\n", value);
}

int dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue is empty\n");
        return -1;
    }
    int value = queue[front];
    front++;
    return value;
}

int main() {
    enqueue(10);
    enqueue(20);
    enqueue(30);

    printf("Dequeued: %d\n", dequeue());
    printf("Dequeued: %d\n", dequeue());

    return 0;
}
```

Q8. Explain Singly Linked List with a node diagram.

A Singly Linked List is a linear data structure made up of nodes, where each node contains two parts: the DATA field (which stores the actual value) and the NEXT field (a pointer/reference to the next node in the sequence). The last node's NEXT field points to NULL, indicating the end of the list.

Unlike an array, a linked list does not require contiguous memory, and elements can be inserted or deleted easily without shifting other elements; however, accessing an element requires traversing the list sequentially from the head, taking $O(n)$ time in the worst case.

Node diagram for a list containing 10, 20, 30:

HEAD

|

v

[10 | *]--> [20 | *]--> [30 | NULL]

Here, each box represents a node with its DATA value and a pointer (*) to the next node. HEAD points to the first node, and the last node's pointer is NULL, marking the end of the list.